

Q&A preparation

- [How does VEREFOO work in detail? What is a MaxSMT problem?](#)
 - [What are VerefooSerializer, VerefooProxy and Translator?](#)
 - [Why only a qualitative test, not a quantitative one?](#)
 - [Other likely questions](#)
 - [Why does the iterative result have one extra firewall / rule vs. the monolithic one?](#)
 - [Why is the iterative approach's optimality only local, not global?](#)
 - [What happens if a new NSR makes the problem UNSAT mid-iteration?](#)
 - [Does it support removing requirements, or only adding them?](#)
 - [How do you handle ties when multiple candidates are equally cheap?](#)
 - [Why CESNET specifically as the test topology?](#)
 - [What's the extra cost/overhead vs. a single batch call?](#)
-

How does VEREFOO work in detail? What is a MaxSMT problem?

- Two inputs: a **Service Graph** (topology — nodes & links, no security info) and a set of **NSRs** (which flows must be allowed/denied)
- From the Service Graph, VEREFOO builds an **Allocation Graph**: every link gets an Allocation Place — a candidate firewall position
- The whole problem is encoded as a **MaxSMT instance**:
 - Each Allocation Place = a boolean decision variable
 - **Hard constraints** = correctness → solver only accepts solutions where every NSR is satisfied
 - **Soft constraints** = optimality (weighted) → minimize number of firewalls and rules
- Solved with the **Z3 SMT solver**
- Hard constraints are non-negotiable → any returned solution is correct by construction (formal correctness guarantee)

What are VerefooSerializer, VerefooProxy and Translator?

- **VerefooSerializer** — entry point; every variant (vanilla/React/iterative) starts here; builds the Allocation Graph, picks the constructor logic
- **VerefooProxy** — classifies NSRs (added/kept/deleted), identifies the reconfigurable area, builds the MaxSMT problem for Z3
- **Translator** — converts Z3's model back into a concrete NFV object (firewall allocation + filtering policies)
- My contribution touches **only VerefooSerializer**: new constructor wraps the existing React-VEREFOO call in a loop, one requirement at a time
- VerefooProxy, the MaxSMT encoding, and the Translator: **unchanged, not a single line touched**

Why only a qualitative test, not a quantitative one?

- Primary goal was to demonstrate **correctness**, not speed — shown by tracing every iteration step by step, checking nothing breaks

- A meaningful quantitative comparison needs **scale**: larger topologies, larger/varied NSR sets, multiple runs for solver variance
 - CESNET (23 nodes, 15 requirements) is realistic but small — both iterative and monolithic solve in a fraction of a second, so a timing comparison here wouldn't be informative
 - Quantitative benchmarking is explicitly the **first item in future work**
-

Other likely questions

Why does the iterative result have one extra firewall / rule vs. the monolithic one?

- Monolithic sees all 15 requirements at once → can find one Allocation Place covering **two** isolation needs together (Ostrava + part of Tabor)
- Iterative commits to Ostrava's placement early, in P0 — long before Tabor's requirement exists
- Can't retroactively relocate → needs a dedicated firewall + one follow-up rule later
- Same firewall count, **one extra ALLOW rule** — direct cost of no global visibility

Why is the iterative approach's optimality only local, not global?

- Each step only reconsiders the area affected by the **one** new requirement — everything else stays fixed
- Keeps each subproblem minimal — that's the whole point
- But: can't reshuffle a decision already committed to in a previous iteration
- Vanilla VEREFOO sees everything at once → can always find the better global arrangement

What happens if a new NSR makes the problem UNSAT mid-iteration?

- Loop does **not** break immediately
- Current configuration (from all previously successful iterations) is kept as the result
- Failure is recorded and surfaced to the caller **after** the loop completes
- No work lost on earlier, successfully satisfied requirements

Does it support removing requirements, or only adding them?

- Only **additions** in this thesis
- React-VEREFOO itself supports a "deleted" category, but the iterative loop only ever passes one **added** requirement per call
- Supporting removals + arbitrary mixes → explicitly listed as future work

How do you handle ties when multiple candidates are equally cheap?

- Currently **nothing explicit** — solver picks based on soft constraints (bias toward reusing existing config)
- Discussed directly in the thesis: a tie can propagate unevenly into later iterations (one candidate might cover a future requirement for free, another might not)
- **Future work**: traffic-aware ordering — anticipate future traffic to break ties and reduce later reconfigurations

Why CESNET specifically as the test topology?

- Real, publicly documented backbone — Czech national academic/research network
- Realistic mix of link capacities, non-trivial structure: central hub (Praha), redundant paths, peripheral nodes
- Enough complexity to exercise reconfiguration, reuse, and ties — without a synthetic topology
- Note: topology and capacities are real; the **NSR set** was constructed for this evaluation, not CESNET's actual policy

What's the extra cost/overhead vs. a single batch call?

- Each solver invocation has a **fixed overhead**: parsing, MaxSMT encoding, solver initialization
- With N requirements one at a time → that fixed cost is paid **N times** instead of once
- Trade-off: minimal problem per step, but overhead accumulates linearly with N
- Quantifying exactly where this trade-off tips is the main open question for the future quantitative benchmark